

רמזים

שים לב

שלוש דרגות בכל קושי. קרא את הדחיפה, נסה עשר דקות, ורק אז את האסטרטגיה. הכמעט-פתרון הוא הרשת, לא הקיצור. הקושי הראשון הוא היחיד שנוגע בחלק א, והוא על שיטה בלבד. אין כאן רמז לאף אחת מחמש התקלות, בכוונה: זה החלק שאתה עושה לבד.

א · "אני תקוע באתגר הניפוי ולא יודע מאיפה להתחיל"

דחיפה. אתה מסתכל בקוד. תסתכל במסך. פתח את הקובץ `debug/js/app.js` בראש והשווה את חמש השורות שכתוב בהן מה היישום אמור לעשות למה שקורה בפועל, אחת-אחת. **רשום את החמש בצד.** שתיים מהן שבורות בצורה שצועקת ושלוש בצורה שותקת, אבל כולן שבורות במסך.

אסטרטגיה. סדר עבודה שעובד גם כשאינן לך מושג:

1. טען. קרא את הקונסולה מלמעלה למטה. כל שורה אדומה היא מתנה — יש בה קישור לשורה.

2. עבור על החמש ברשימה שלך ובדוק אחת-אחת: **מה אני מצפה לראות, ומה אני רואה.**

3. לכל פער — לפני שאתה נוגע בקוד — כתוב משפט אחד: "אני חושב ש-X הוא Y במקום Z".

4. עכשיו לך למצוא את X. **נקודת עצירה, לא הדפסה,** כי היא מראה את כל המשתנים בבת אחת.

כמעט-פתרון. שני מספרים שכדאי להשוות בכל יישום רשימות: כמה פריטים יש ב**נתונים** (`console.table` על המערך בקונסולה) וכמה שורות יש **על המסך**. אם הם לא שווים, מצאת תקלה בלי לקרוא שורת קוד אחת. עשה את זה ראשון — זה לוקח עשר שניות.

ב · "כלום לא קורה. העמוד ריק ואין שגיאה"

דחיפה. מה מחזירה `itemRow`? מה עושה `render`? פונקציה שגופה ריק מחזירה `undefined` ולא זורקת כלום — היא פשוט לא עושה שום דבר, בשקט.

אסטרטגיה. התחל מ-`render` ולא מ-`itemRow`. שים בתוכה שורה אחת —

`console.log('render', items.length)` — ורענן. אם המספר מודפס, הפונקציה נקראת והבעיה בגוף שלה. אם לא, השורה האחרונה בקובץ לא רצה.

כמעט-פתרון. `render` היא ארבעה דברים בסדר הזה, ואף אחד מהם אינו יותר משורה או שתיים: לרוקן `(list.replaceChildren())`, למלא (לולאה עם `(list.append(itemRow(item)))`), להחליט אם המצב הריק מוצג `(empty.hidden = items.length > 0)`, ולעדכן את המספר `(count.textContent = items.length)`.

ג · "אני בונה את השורה ולא רואה אותה"

דחיפה. בנית אלמנט. חיברת אותו למשהו?

אסטרטגיה. `document.createElement('li')` יוצר אלמנט **שאינו בעמוד**. הוא קיים בזיכרון וזהו. הוא נכנס לעמוד רק כשמישהו קורא ל-`append` עליו או על מי שמכיל אותו. בדוק בקונסולה: `el.isConnected` מחזיר `false` על אלמנט מנותק.

כמעט-פתרון. `itemRow` מחזירה את ה-`<li>` ולא מחברת אותו לכלום — זו העבודה של `render`. אם שכחת את `return` בסוף `render`, `itemRow` תקבל `undefined` ותנסה לצרף אותו, וזה גם המקום שבו תראה שגיאה.

ד · "הכותרת של פריט מופיעה כטקסט עם תגיות"

דחיפה. `textContent` שם את המחרוזת **כטקסט**. זה בדיוק מה שאנחנו רוצים. אם אתה רואה תגיות על המסך, בנית מחרוזת HTML איפשהו במקום להשתמש בשלושה אלמנטים.

אסטרטגיה. אל "תתקן" את זה עם `innerHTML`. הכותרות מגיעות מטופס, ומחרוזת שמשתמש הקליד היא בדיוק זו שאסור שתגיע למפרסר. בנה שלושה אלמנטים — `span`, `span`, `button` — וצרף אותם ל-`li` באמצעות `li.append(a, b, c)`.

כמעט-פתרון. `append` מקבל כמה ארגומנטים בבת אחת, וגם מחרוזות (שהוא הופך לטקסט בטוח). המחלקות הן `item-title` על הראשון ו-`remove` על הכפתור; שניהם קבועים כי הבודק מחפש אותם.

ה · "ההסרה עובדת רק לפעמים"

דחיפה. לחץ בדיוק על **המילה** "הסר", ואז בדיוק על **השוליים** של הכפתור. שתי תוצאות שונות?

אסטרטגיה. `event.target` הוא האלמנט שהעכבר פגע בו — ה-`span`, לא הכפתור. `event.currentTarget` הוא זה שהמאזין רשום עליו. הדפס את שניהם בשורה הראשונה של המאזין ולחץ פעמיים, בשני המקומות.

כמעט-פתרון. אל תבדוק את ה-`target`. **טפס ממנו:**

```
const button = event.target.closest('button.remove');
if (!button) return;
```

`closest` הולך למעלה מהאלמנט שנלחץ עד שהוא מוצא התאמה — ולכן זה עובד גם אם מחר תשים אייקון בתוך הכפתור.

ו · "ההסרה עובדת על השורות הראשונות ולא על החדשות"

דחיפה. מתי רשמת את המאזינים, ומתי נוצרו השורות החדשות?

אסטרטגיה. לולאה על `querySelectorAll('.remove')` רצה **פעם אחת**, על האלמנטים שהיו קיימים באותו רגע. מאזין נקשר לאלמנט, לא לבורר. שורה שנוצרה אחר כך לא מכירה אף אחד. וגרוע מזה: אחרי `render` הבא, **כל** הכפתורים חדשים והרשימה כולה מתה.

כמעט-פתרון. מאזין אחד, על `#list`, שנרשם פעם אחת בטעינה. `render` מרוקן את הילדים של `#list` ולא מחליף את ה-`<ul>` עצמו, ולכן המאזין שורד כל רינדור. בפנים, `closest` מהקושי הקודם.

ז · "אני לוחץ הוסף והעמוד נטען מחדש"

דחיפה. הכתובת בשורת הכתובת השתנתה? יש בה סימן שאלה?

אסטרטגיה. זו התנהגות ברירת המחדל של טופס, מ-1995: לארוז את השדות ל-URL ולבקש את העמוד מחדש. הכול שבזיכרון נמחק. `event.preventDefault()` בשורה הראשונה של המטפל עוצר את זה.

כמעט-פתרון. ושים לב על **מה** רשמת את המאזין: `submit` על ה-`<form>`, לא `click` על הכפתור. הראשון תופס גם Enter מתוך שדה טקסט; השני לא, וזה נשבר אצל מי שמקליד מהר.

ח · המספר מתנהג מוזר – 20241 במקום 2025

דחיפה. מה הטיפוס של `input.value`? בדוק בקונסולה, לא בראש.

אסטרטגיה. `input.value` הוא **תמיד מחזורת**, גם כש-`type="number"`. `"2024" + 1` הוא `"20241"`. מה שמבלבל: `"2024" > 1000` הוא `true`, כי אופרטור השוואה כן ממיר – ולכן הבאג עובר בדיקה ידנית.

כמעט-פתרון. המרה אחת, **בגבול** – בשורה שבה הערך נכנס למערכת:

```
const number = Number(numberField.value);
```

מכאן והלאה זה מספר, ואף פונקציה אחרת לא צריכה לזכור את זה.

ט · שדה ריק עובר את הוולידציה

דחיפה. מה מחזירה `Number('')`?

אסטרטגיה. `0`. לא `NaN`. אז `Number.isInteger(Number(''))` הוא `true`, וכל בדיקה מהצורה "האם זה מספר" עוברת על שדה שאף אחד לא מילא.

כמעט-פתרון. בדוק את הריקנות על **המחזורת**, לפני ההמרה:

```
if (numberField.value.trim() === '' || !Number.isInteger(number))
```

שני התנאים, בסדר הזה.

י · המצב הריק לא נעלם / לא מופיע

דחיפה. מי מחליט אם הוא מוצג – המטפל שהוסיף, המטפל שהסיר, או `render`?

אסטרטגיה. אם התשובה היא "המטפלים", יש לך שני מקומות שצריכים לזכור את אותו כלל, והם יפרדו. הכלל הוא נגזרת של הנתונים: אם המערך ריק, המצב הריק מוצג. זו שורה אחת, בתוך `render`, ואז היא לא יכולה להיות שגויה.

כמעט-פתרון. `empty.hidden = items.length > 0;` – מאפיין, לא מחלקה, כי `hidden` הוא מה שה-HTML כבר מספק. ובדוק את זה: רוקן את `js/items.js` למערך ריק ורענן.

יא · "aria-invalid" נשאר אדום גם אחרי שתיקנתי

דחיפה. איפה אתה מנקה אותו?

אסטרטגיה. סימון קורה בכישלון, וניקוי צריך לקרות **בכל ניסיון**. לפני שבודקים משהו. אחרת המצב של הניסיון הקודם נשאר על המסך.

כמעט-פתרון. השורות הראשונות של `readForm` מאפסות את שני השדות ל-`'false'`, ורק אחר כך מתחילות הבדיקות. אותו דבר ל-`#form-error`: הוא מתרוקן בהצלחה.

יב · "מה זה 'מטפל לא נוגע ב-DOM ישירות'?"

דחיפה. אחרי שהסרת פריט, כמה מקומות בקוד יודעים שהוא נעלם?

אסטרטגיה. אם המטפל עושה `row.remove()` **וגם** מסנן את המערך, יש שני מקורות אמת והם יפרדו – לא היום, בשבוע 10, כשתוסיף שמירה. אם הוא רק מסנן את המערך וקורא ל-`render`, יש אחד.

כמעט-פתרון. גוף המאזין כולו הוא שלוש שורות: למצוא את הכפתור, לסנן את `items`, לקרוא ל-`render()`. אין בו `remove()`, אין `classList`, ואין `textContent`. זו בדיקת הצורה ששבוע 9 נותן לה שם.